

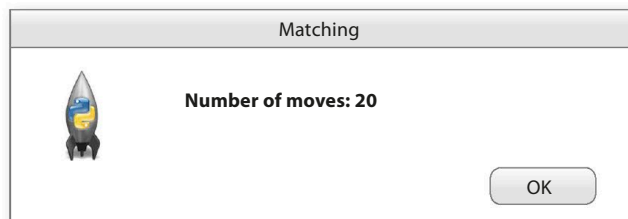
4 Count the moves

A move is two button presses (one matching attempt). So you only need to add 1 to the **moves** variable when the **show_symbol()** function is called for the first or the second button press – not for both. Let's do it for the first button press. Change the **show_symbol()** function to look like this.

```
if first:
    previousX = x
    previousY = y
    first = False
    moves = moves + 1
```

5 Display a message

Now add the following code near the bottom of the **show_symbol()** function. It will track the matched pairs and show a message box at the end of the game telling the player how many moves they took. When the player clicks the box's OK button, the code calls the **close_window()** function, which we'll add next.



```
buttons[x, y]['command'] = DISABLED
```

```
pairs = pairs + 1
```

```
if pairs == len(buttons)/2:
```

```
    messagebox.showinfo('Matching', 'Number of moves: ' +
                        str(moves), command=close_window)
```

Add 1 to the number of pairs found.

This line displays a box showing the number of moves.

If all the pairs have been found, run the code under this line.

△ How it works

There are 12 pairs of symbols, so you could simply have typed **pairs == 12** in the hack. However, your code is smarter than this. It calculates the number of pairs by using **pairs == len(buttons)/2**. This allows you to add more buttons to the game without having to update this bit of code.

6 Close the window

Finally, you need to create a **close_window()** function, to make the program exit the game when the player clicks the OK button on the "Number of moves" message box. Add this code under the line that imports the modules.

```
def close_window(self):
    root.destroy()
```

This command closes the window.